

Ejercicio_Preparacion_Para _El_Parcial — Programacion 1

Sistema de Gestion de Clinica Veterinaria

Contexto del Problema

Una clinica veterinaria de barrio necesita digitalizar el registro de sus pacientes para mejorar la atencion y evitar la perdida de informacion. Actualmente llevan el control de forma manual en cuadernos y requieren un programa que permita gestionar las mascotas atendidas y la cantidad de turnos pendientes que tiene cada una registrados en el sistema.

Requerimientos Tecnicos (Restricciones Estrictas)

El codigo debe respetar las siguientes limitaciones tecnicas. El incumplimiento de estas normas resultara en una calificacion maxima de 10% (1.0):

Estructuras permitidas: Estructuras secuenciales, condicionales (if, elif, else), bucles (while, for), estructura de seleccion para el menu y definicion de funciones con def.

Estructura de Datos: La unica estructura de almacenamiento permitida sera una lista de diccionarios llamada pacientes[]. Cada elemento debera tener exactamente las claves mascota y turnos. No se permite utilizar listas paralelas, diccionarios externos, sets, clases ni estructuras avanzadas.

Modularizacion obligatoria: Cada opcion del menu debe resolverse dentro de una funcion dedicada con def. No se aceptaran programas sin modularizar.

Manejo de errores: Uso correcto de try/except para capturar entradas invalidas. Se permite raise para errores de reglas de negocio.

Prohibiciones: No se pueden usar variables globales. Si una funcion necesita pacientes[], debe recibirla como parametro.

Funcionalidades del Menu

El programa debe presentar un menu interactivo persistente con las siguientes opciones:

1. Registro Inicial de Mascotas

El sistema pregunta cuantas mascotas se van a registrar y, por cada una, solicita inmediatamente el nombre de la mascota y la cantidad de turnos pendientes. Solo puede ejecutarse cuando pacientes[] esta vacia. Si ya hay mascotas registradas, informa que para agregar nuevas debe usarse la opcion 5. Si un dato es invalido (nombre vacio, duplicado o turnos negativos), se informa el error y se vuelve a pedir ese dato hasta completarlo correctamente, respetando la cantidad total indicada al inicio.

2. Visualizacion de Pacientes

Muestra el listado completo de mascotas registradas junto a la cantidad de turnos pendientes actuales, recorriendo la lista pacientes[].

3. Consulta de Turnos

Busca una mascota por su nombre y muestra cuantos turnos pendientes tiene registrados. Informa claramente si la mascota no se encuentra en el sistema.

4. Reporte de Mascotas sin Turnos

Lista unicamente las mascotas que tengan cero turnos pendientes, es decir, aquellas que ya completaron toda su atencion o aun no tienen turno asignado.

5. Alta de Nueva Mascota

Permite registrar una unica mascota nueva al final de pacientes[]. Si el nombre esta vacio, ya existe en el sistema, o la cantidad inicial de turnos es negativa, se lanza un error con mensaje descriptivo y se vuelve al menu principal sin agregar nada.

6. Actualizacion de Turnos (Asignacion / Atencion)

Permite modificar la cantidad de turnos pendientes de una mascota existente:

- **Atencion:** disminuye los turnos pendientes al registrar que una mascota fue atendida. Debe validarse que haya al menos un turno pendiente (no se puede registrar una atencion si hay cero turnos).
- **Asignacion:** aumenta los turnos al reservar un nuevo turno para la mascota.

7. Salir

Finaliza la ejecucion del sistema.

Criterios de Validacion (Robustez)

El sistema debe ser a prueba de errores comunes. Cada regla de negocio debe estar protegida por try/except:

- No permitir nombres de mascotas vacios ni duplicados (lanzar ValueError con mensaje descriptivo).
- No operar sobre el sistema si pacientes[] esta vacia: informar claramente en opciones 2, 3, 4 y 6.
- Validar con try/except que las cantidades ingresadas sean numeros enteros.
- Impedir registrar una atencion cuando la mascota no tenga turnos pendientes (lanzar ValueError).

- Informar claramente si se intenta operar sobre una mascota que no existe en el sistema.
- Gestionar con try/except las opciones invalidas del menu (entradas no numericas o fuera de rango).
- **Comparacion de nombres:** Ignorar mayusculas/minusculas y espacios iniciales o finales. Por ejemplo, "Firulais", "firulais" y " FIRULAIS " deben considerarse la misma mascota.
- **Validacion de cantidades:** La cantidad de mascotas a registrar debe ser un entero mayor que cero. Los turnos iniciales pueden ser cero o mas, nunca negativos. Las cantidades para atencion o asignacion deben ser enteros mayores que cero.
- **Sistema vacio:** Si pacientes[] esta vacia, las opciones 2, 3, 4 y 6 deben informar que no hay mascotas registradas, sin producir errores de ejecucion.

Estructura Minima Esperada

El siguiente esquema ilustra la organizacion modular requerida. No es el unico diseno valido, pero si la referencia de minima aceptable:

```
def registrar_mascotas(pacientes): ...
def mostrar_pacientes(pacientes): ...
def consultar_turnos(pacientes): ...
def reporte_sin_turnos(pacientes): ...
def alta_mascota(pacientes): ...
def actualizar_turnos(pacientes): ... # atencion / asignacion

# Bloque principal
pacientes = [] # lista de dicts {'mascota': str, 'turnos': int}
opcion = 0
while opcion != 7:
    try:
        opcion = int(input('Seleccione una opcion: '))
        if opcion == 1: registrar_mascotas(pacientes)
        elif opcion == 2: mostrar_pacientes(pacientes)
        # ... etc.
    except ValueError as e:
        print(f'Error: {e}')
```